Portable Monitoring temperature & humidity for Safety

Team Members:

Nahid & Rohil

Project Description (Purpose)

The purpose of our project is to provide cheap solution to small grocery stores, small office or even houses to monitor Temperature and Humidity and display message through LED matrix.

Project Description (Business Problem)

Many indoor environments, such as stores, student buildings, labs, apartments, and workplaces, lack **real-time environmental monitoring**. Poor ventilation, rising humidity, and unstable pressure can cause:

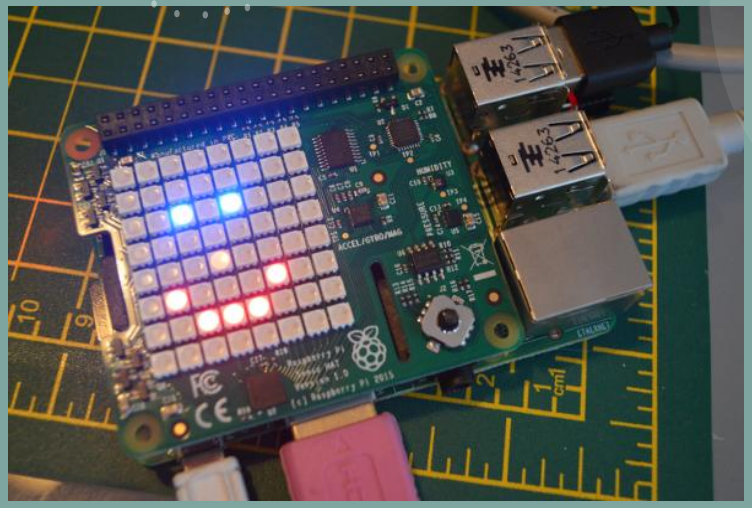
- discomfort
- heat stress
- breathing problems
- mold risk
- HVAC malfunction

So, we have a small solution that uses Raspberry Pi and sense that (sensor) which is used to monitor Temperature and Humidity.

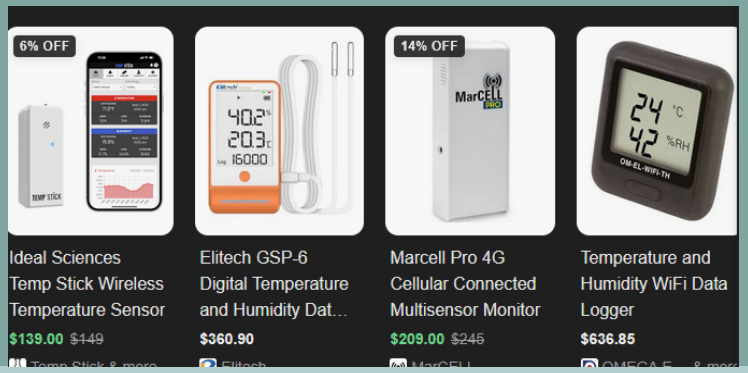
Continue Next Slide

Impact & Beneficiaries

Price compare with our product and other products in market.



Our monitoring system costs around total **$80-$90** whereas other market products cost **$360-$600**



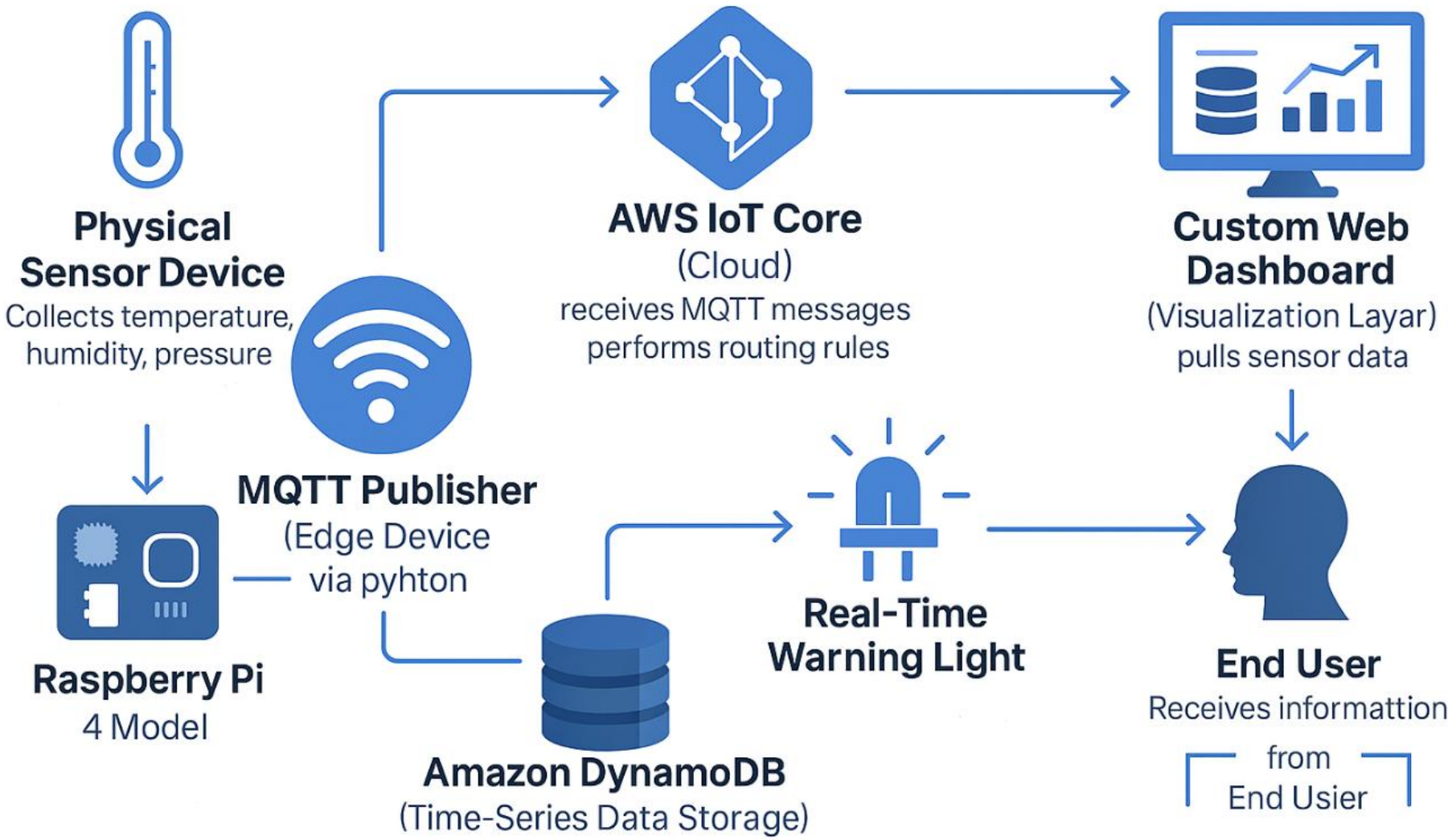
Innovation

- Cheap solution for small businesses and house to monitor Temperature and Humidity.
- Portable you can carry anywhere, no need any more accessories like monitor, HDMI cables, keyboard or mouse.
- Can be controlled through your phone with secure VNC. So, No data leaks in between.
- The collected data is sent to DynamoDB and can be downloaded for further analysis.
- It can be fully customized according to customer needs as we made this system from Scratch with **Python**.

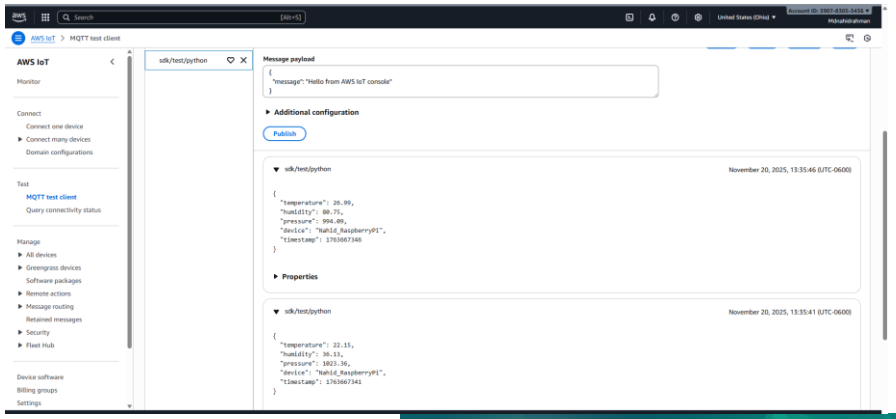
Innovation

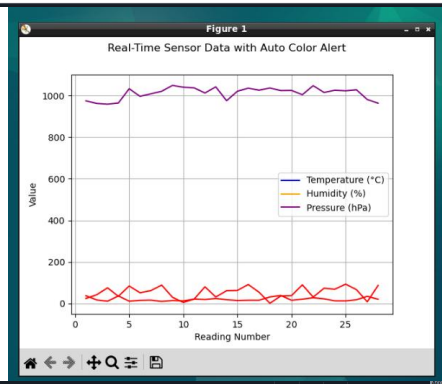
Portable : Totally Controlled by Phone

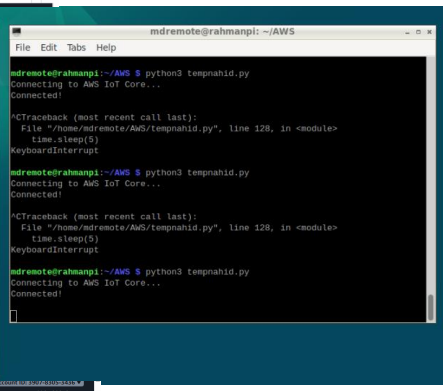
Avatar

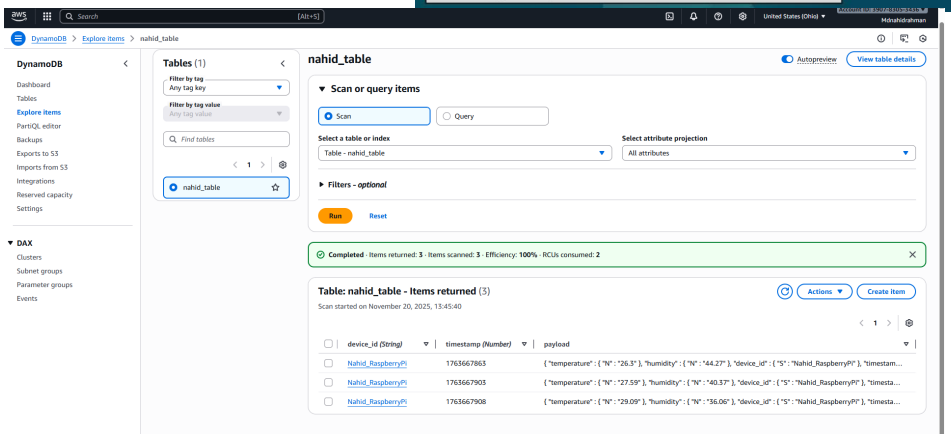

Physical
Sensor Device
Collects temperature,
humidity, pressure
AWS IoT Core
(Cloud)
receives MQTT messages
performs routing rules
Custom Web
Dashboard
(Visualization Layar)
pulls sensor data
MQTT Publisher
(Edge Device
via pyhton
Raspberry Pi
4 Model
Real-Time
Warning Light
End User
Receives informattion
from
End Usier
Amazon DynamoDB
(Time-Series Data Storage)

Physical Implementation









MIT IoT Model: Sensors

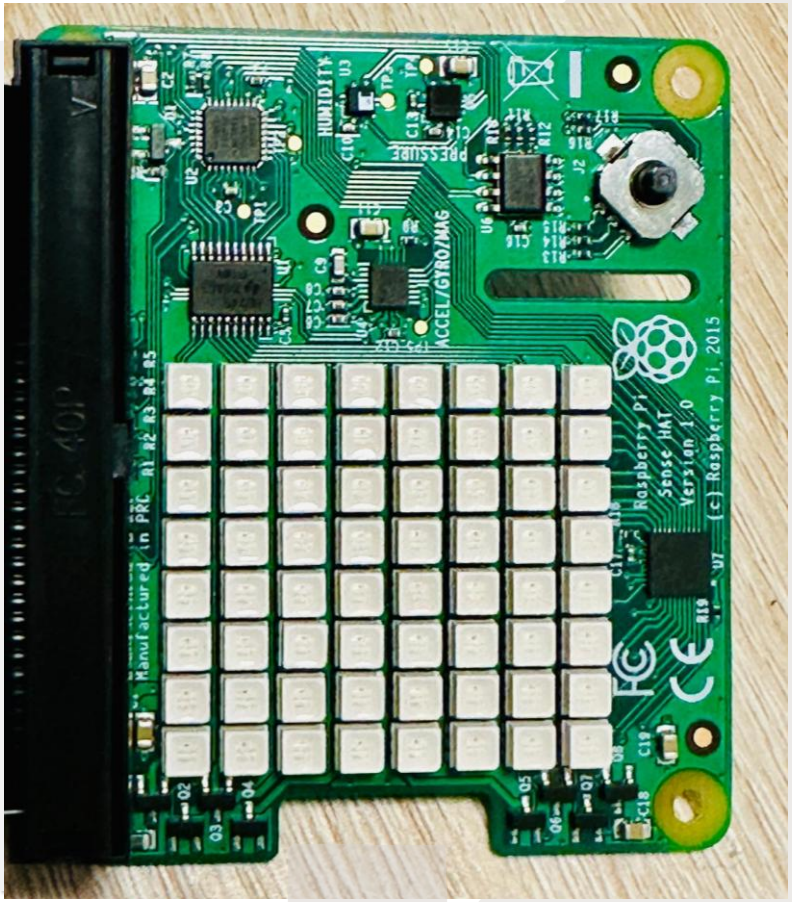
Sensors Used: Sense Hat

Collects:

- **Temperature data**→ °C
- **Humidity data**→ %RH

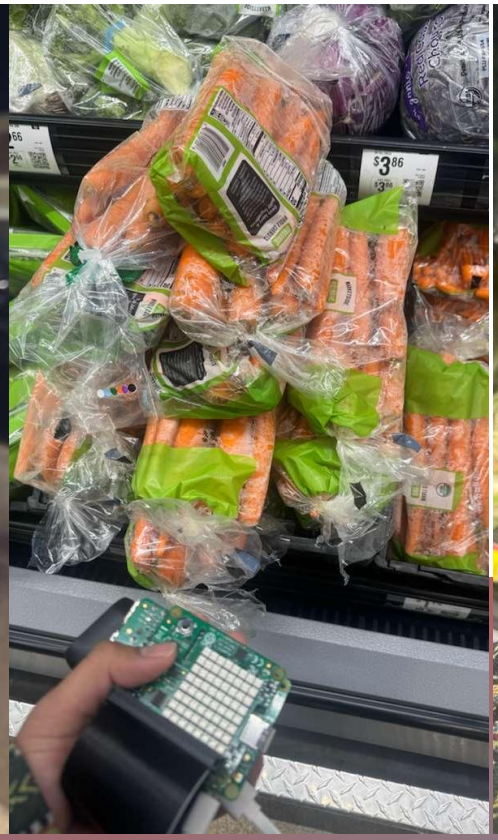
Data Collected:

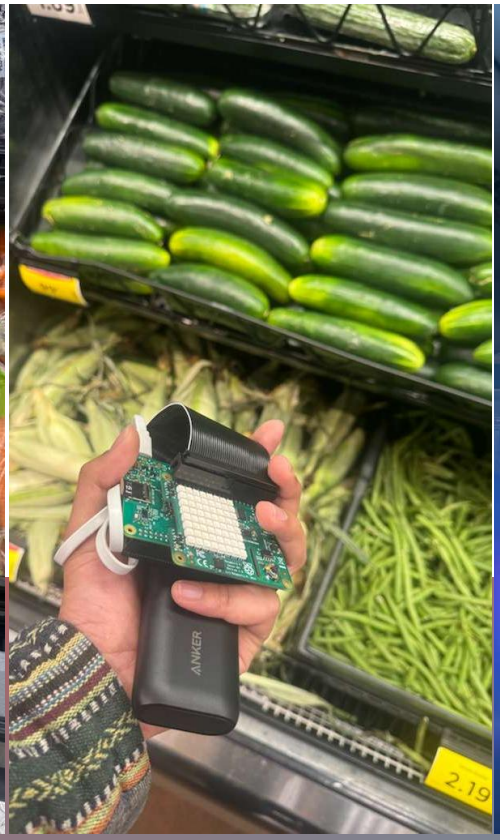
- Environmental readings every 5 seconds
- Multiple place in Jonesboro
- Data forwarded through AWS IoT for storage and analysis



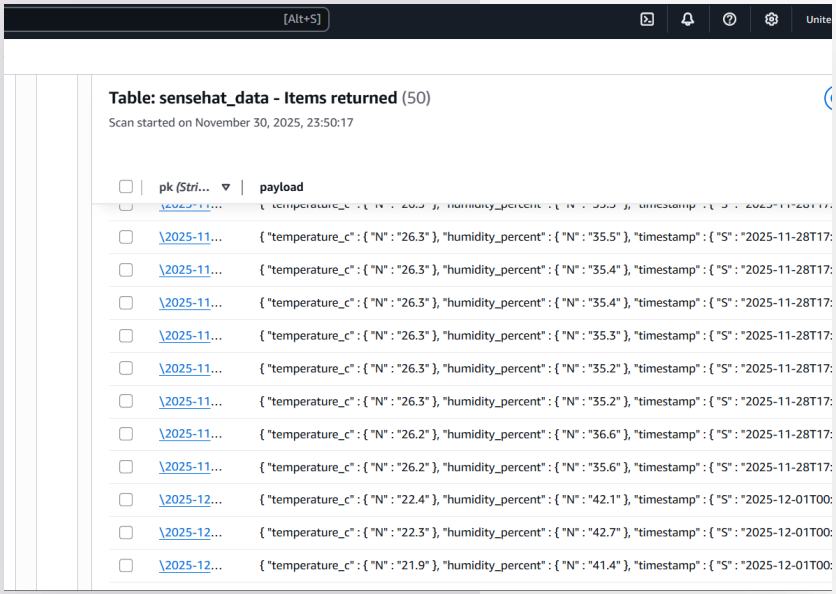
Continue Next Slide

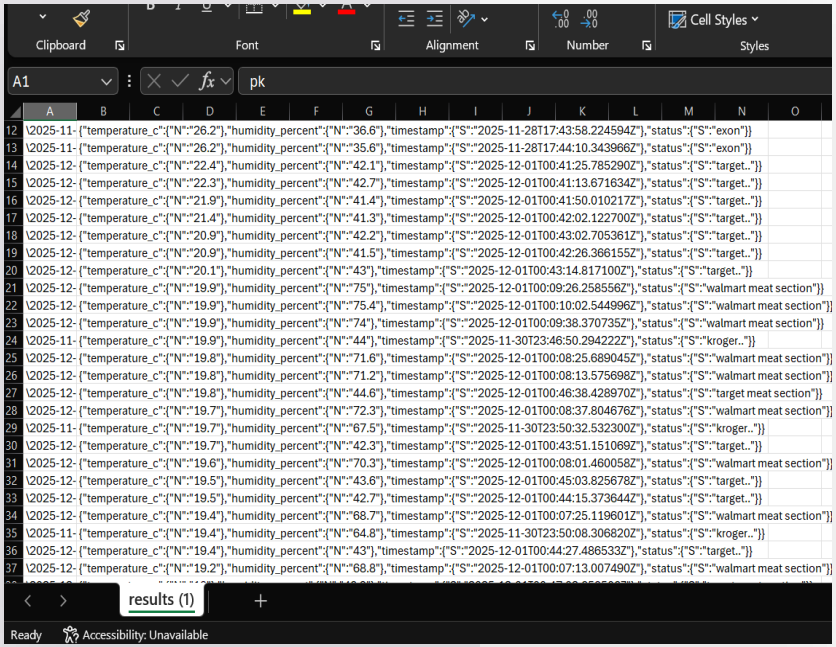
MIT IoT Model: Sensors





We went to different grocery store in town to make sure our system is working and uploading data to DynamoDB.





Sample data collected from the **Sense Hat** Sensor from different grocery stores across the town. First picture shows the stored at DynamoDB. Second picture shows data been downloaded as CSV file for analyzing data.

MIT IoT Model: Communication

Communication Method:

- **Wi-Fi network or Cellular** connectivity from the Raspberry Pi to AWS.
- **MQTT protocol** for lightweight, real-time message transfer
- Using Python programming language, It collects data and uploads to IOT core of AWS.
- **AWS IoT Core endpoint** for cloud ingestion
- **X.509 device certificates** for secure authentication and encrypted communication

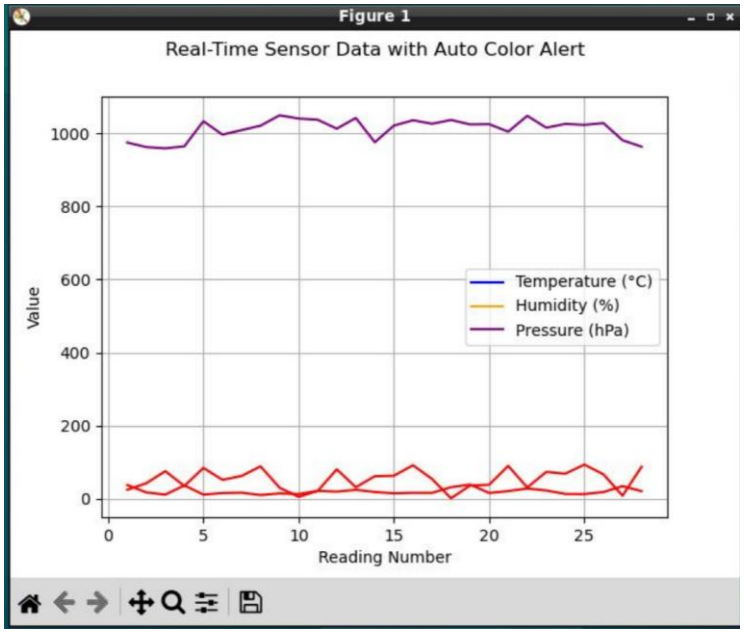
MIT IoT Model: Analytics

Analytics Performed:

- Real time Stored data in DynamoDB can be downloaded in csv and analyze in Excel or even python.

- With python we can see the real time chart which is necessary for monitoring critical environment.

	A	B	C	D	E	F	G	H	I	J	K
1	pk	payload									
2	\2025-11-28T17:36:40.660431	{"temperature_c":{"N":"26.4"},"humidity_percent":{"N":"35.6"},"timestamp":{"S":"2025-11-28T17:36:40.660431Z"},"status":{"S":"walmart"}}									
3	\2025-11-28T17:38:26.793713	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.3"},"timestamp":{"S":"2025-11-28T17:38:26.793713Z"},"status":{"S":"kroger"}}									
4	\2025-11-28T17:40:53.617927	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.2"},"timestamp":{"S":"2025-11-28T17:40:53.617927Z"},"status":{"S":"natural grocery"}}									
5	\2025-11-28T17:41:29.978286	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.4"},"timestamp":{"S":"2025-11-28T17:41:29.978286Z"},"status":{"S":"natural grocery"}}									
6	\2025-11-28T17:44:10.343966	{"temperature_c":{"N":"26.2"},"humidity_percent":{"N":"35.6"},"timestamp":{"S":"2025-11-28T17:44:10.343966Z"},"status":{"S":"exon"}}									
7	\2025-11-28T17:43:58.224594	{"temperature_c":{"N":"26.2"},"humidity_percent":{"N":"36.6"},"timestamp":{"S":"2025-11-28T17:43:58.224594Z"},"status":{"S":"exon"}}									
8	\2025-11-28T17:39:51.634267	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.4"},"timestamp":{"S":"2025-11-28T17:39:51.634267Z"},"status":{"S":"kroger"}}									
9	\2025-11-28T17:42:45.519749	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.7"},"timestamp":{"S":"2025-11-28T17:42:45.519749Z"},"status":{"S":"exon"}}									
10	\2025-11-28T17:38:51.030402	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.2"},"timestamp":{"S":"2025-11-28T17:38:51.030402Z"},"status":{"S":"kroger"}}									
11	\2025-11-28T17:35:52.193499	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.5"},"timestamp":{"S":"2025-11-28T17:35:52.193499Z"},"status":{"S":"walmart"}}									
12	\2025-11-28T17:38:14.674738	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.5"},"timestamp":{"S":"2025-11-28T17:38:14.674738Z"},"status":{"S":"kroger"}}									
13	\2025-11-28T17:38:04.017074	{"temperature_c":{"N":"26.4"},"humidity_percent":{"N":"35.7"},"timestamp":{"S":"2025-11-28T17:38:04.017074Z"},"status":{"S":"target"}}									
14	\2025-11-28T17:37:03.441094	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.2"},"timestamp":{"S":"2025-11-28T17:37:03.441094Z"},"status":{"S":"target"}}									
15	\2025-11-28T17:44:34.575660	{"temperature_c":{"N":"26.2"},"humidity_percent":{"N":"35.4"},"timestamp":{"S":"2025-11-28T17:44:34.575660Z"},"status":{"S":"exon"}}									
16	\2025-11-28T17:43:09.756989	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.9"},"timestamp":{"S":"2025-11-28T17:43:09.756989Z"},"status":{"S":"exon"}}									
17	\2025-11-28T17:37:15.562515	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"36"},"timestamp":{"S":"2025-11-28T17:37:15.562515Z"},"status":{"S":"target"}}									
18	\2025-11-28T17:37:51.921160	{"temperature_c":{"N":"26.2"},"humidity_percent":{"N":"36.5"},"timestamp":{"S":"2025-11-28T17:37:51.921160Z"},"status":{"S":"target"}}									
19	\2025-11-28T17:42:33.402182	{"temperature_c":{"N":"26.3"},"humidity_percent":{"N":"35.4"},"timestamp":{"S":"2025-11-28T17:42:33.402182Z"},"status":{"S":"exon"}}									
20	\2025-11-28T17:36:28.544847	{"temperature_c":{"N":"26.2"},"humidity_percent":{"N":"35.5"},"timestamp":{"S":"2025-11-28T17:36:28.544847Z"},"status":{"S":"walmart"}}									



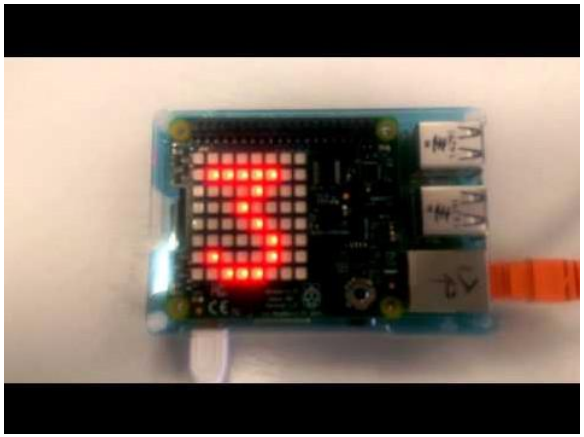
MIT IoT Model: Actuators

Actuator Used:

Sense HAT display "customized messages" through LED Matrix

How It Works:

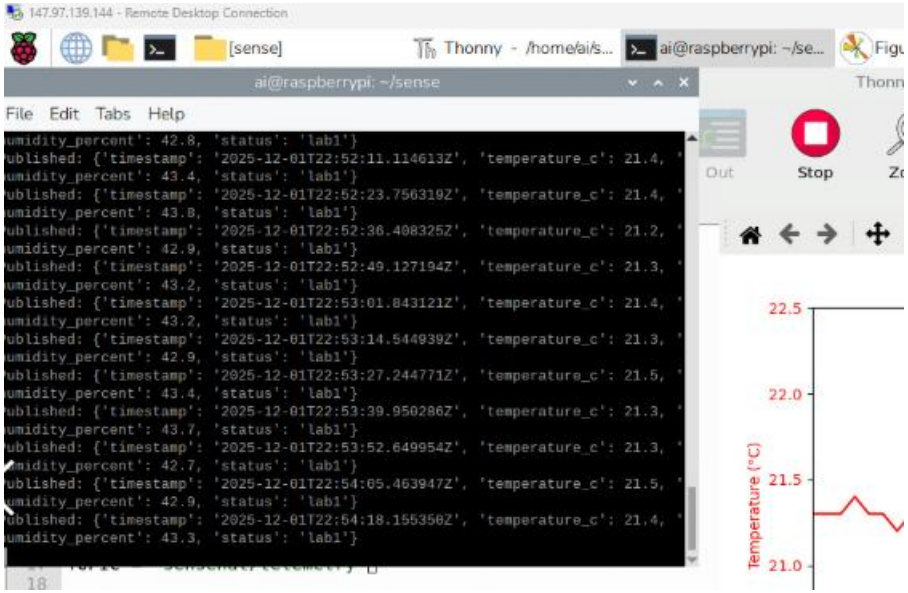
- **"All ok" :** Indicates that all environmental conditions are within the safe range.
- **"Cold" :** Turns on when temperature, humidity, or pressure move outside normal thresholds.
- **Purpose:** Provides immediate, local visual alerts so users can quickly recognize unsafe environmental changes.



Data Collected

Sensor Data Includes:

- Temperature (°C)
- Humidity (%)
- Timestamp
- Location (manually)



JSON Payload Example


```
{
 "temperature": 26.99,
 "humidity": 80.75,
"device": "Nahid_RaspberryPi",
 "timestamp": 1763667346
}
```

Reading Frequency

Frequency: Every 5 seconds

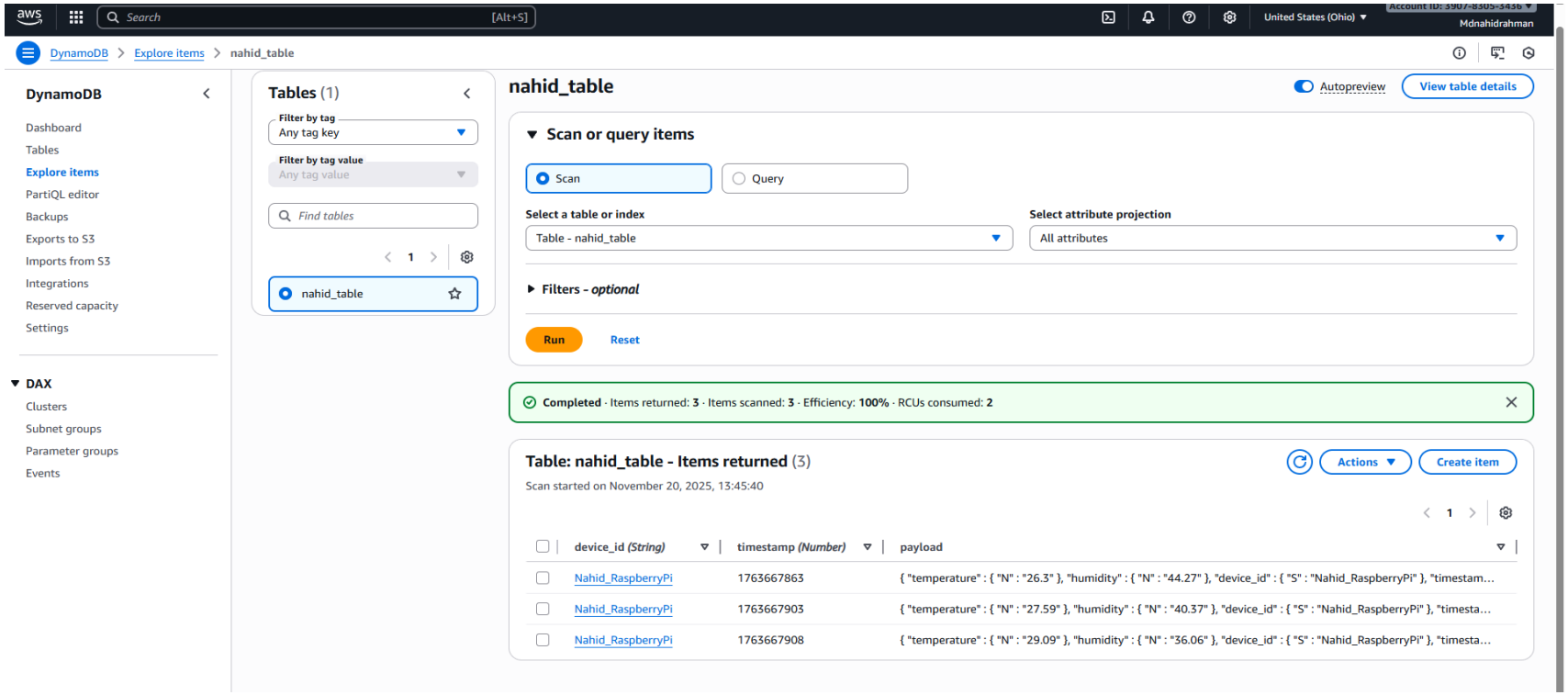
Why 5 seconds?

- **Real-time updates** – fast enough for safety
- **Stable system** – avoids sensor & network overload
- **Lower AWS cost** – fewer writes/messages
- **Clean data** – no unnecessary noise
- **Good analytics** – enough resolution for trends
- **Long-term storage friendly** – manageable data size

SQL Rule + DynamoDB Storage

How the rule works:

- **Listens to the MQTT topic** where our Raspberry Pi publishes data.
- **Uses a SQL statement** (e.g., SELECT * FROM 'sdk/test/python') to read the JSON fields.
- **Checks the message format** to ensure required values like device_id and timestamp are present.
- **Routes the data** into our DynamoDB table (**nahid_table**) when the conditions



SQL Rule + DynamoDB Storage

How data is stored:

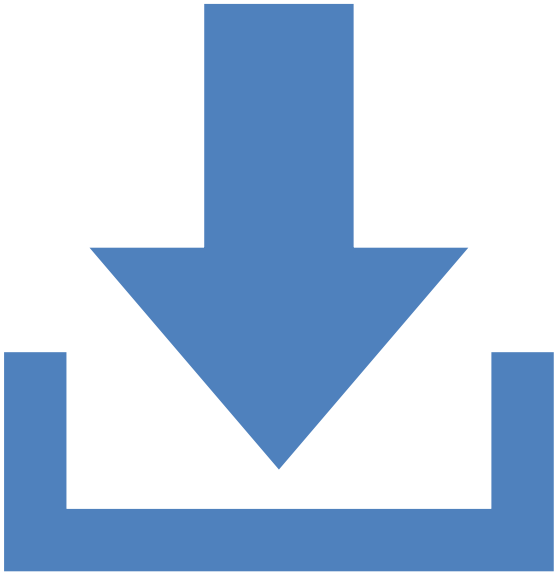
- Each message becomes **one item** in the table.
- **device_id = Partition Key**
- **timestamp = Sort Key**

All sensor values (temperature, humidity, pressure, etc.) are stored in the payload section.

Data Download Format

We export data as:

- **CSV** (easy for Excel analysis)
- **JSON** (easy for Python)



Data Cleaning & Analysis Approach

Steps:

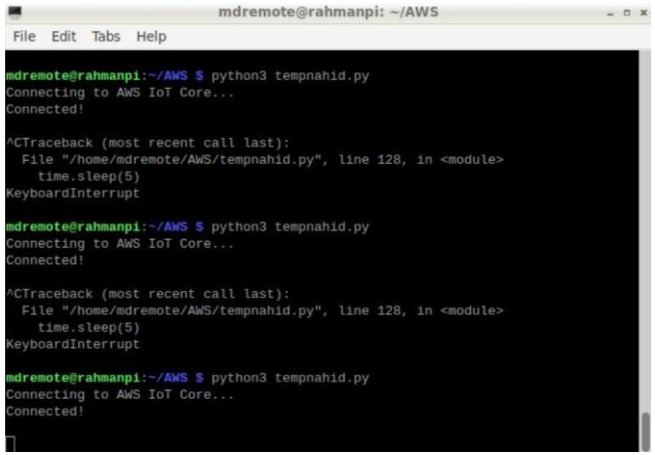
- Export CSV from DynamoDB
- Clean missing values
- Plot temperature, humidity, pressure trends
- Compare locations
- Highlight unsafe values in **red**
- Generate insights
- **Tools used:** Python, Matplotlib

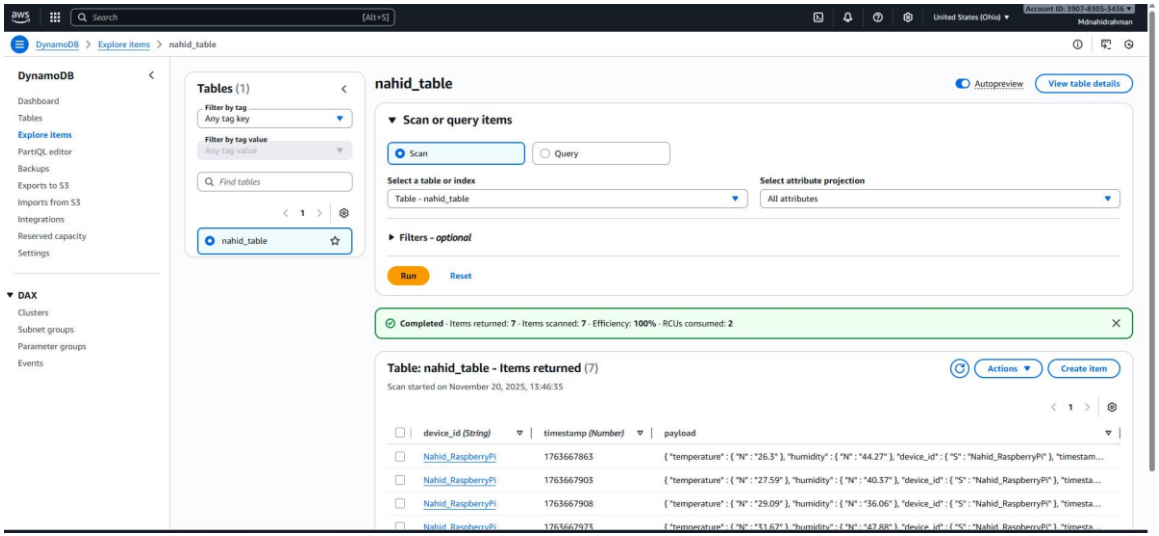
Insights for Safety Decisions

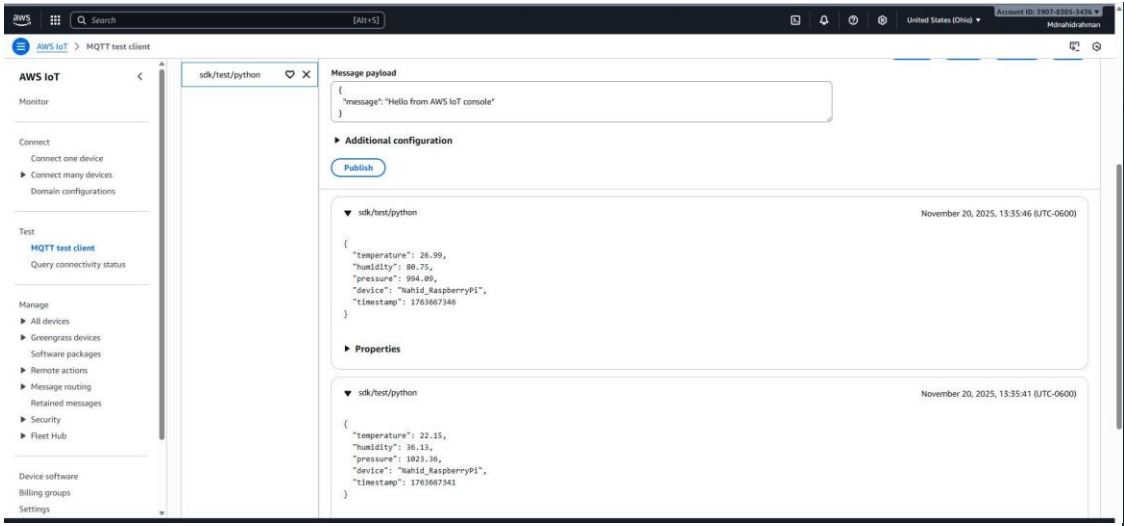
Our analytics help:

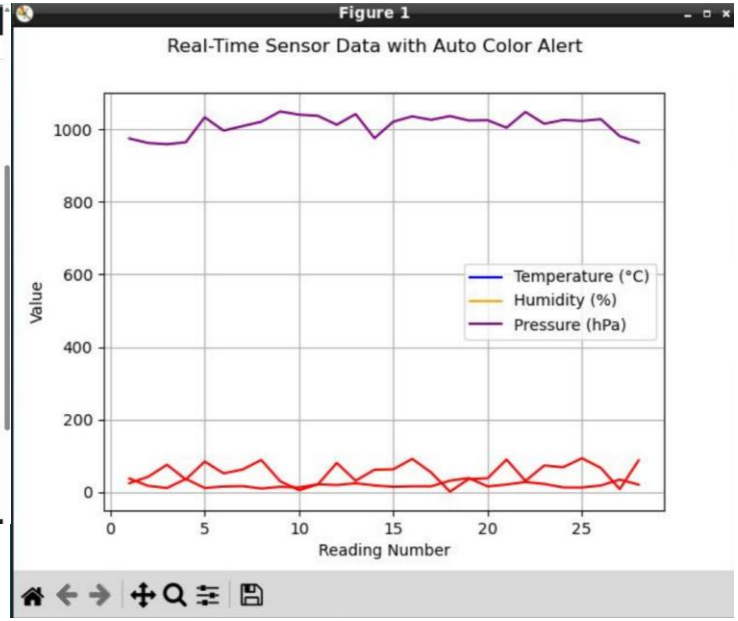
- Identify poorly ventilated areas
- Detect humidity that may cause mold
- Find overheating zones
- Spot pressure instability
- Suggest safer locations for students
- Help building managers fix HVAC issues

System Demonstration

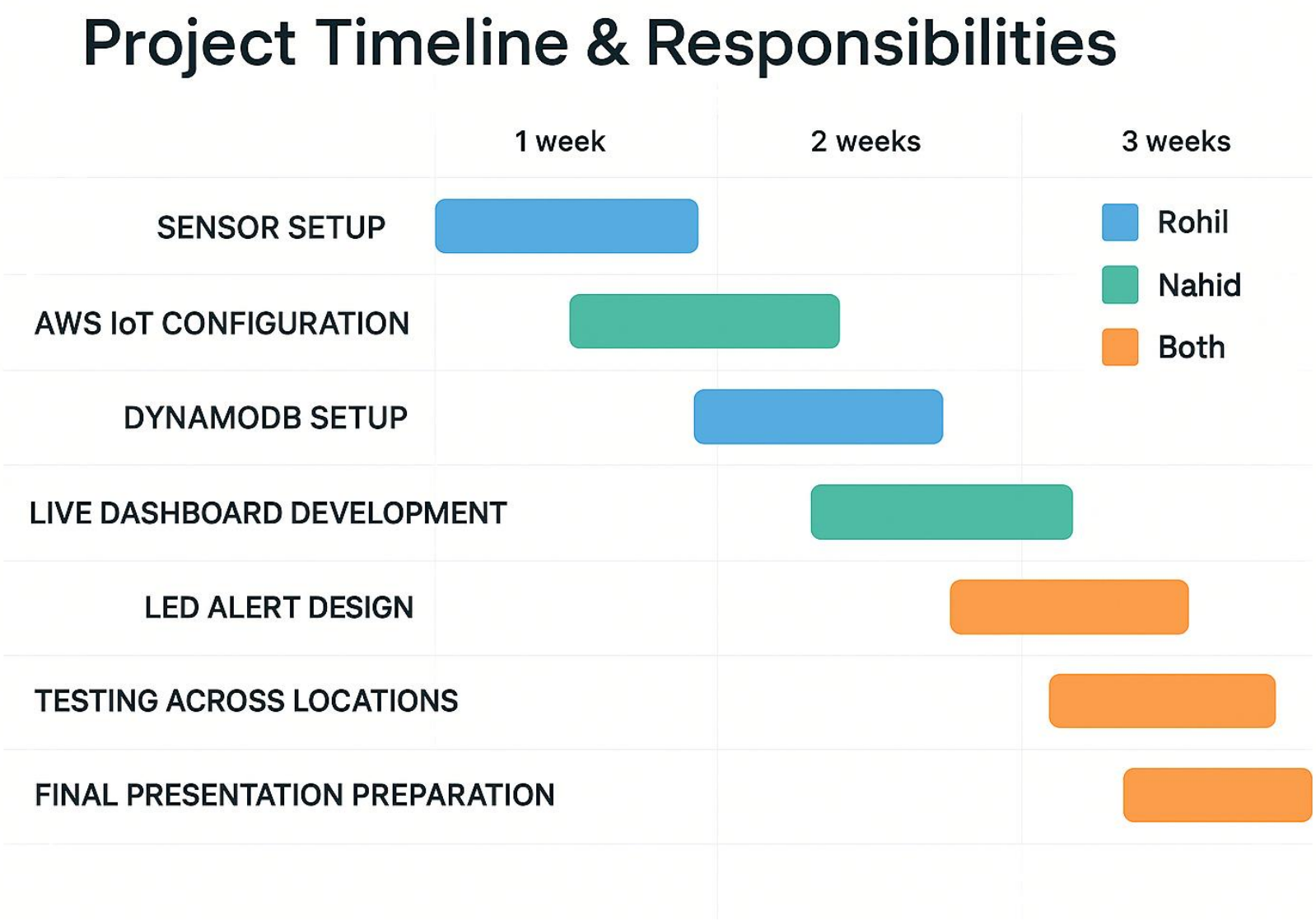








Project Timeline & Responsibilities



Conclusion

- Created full IoT pipeline (sensor → cloud → dashboard → alerts)
- Built a safety-focused system
- Successfully monitored 7 different locations
- Implemented real-time alerts
- Stored historical data for analysis

Future improvements:

- Add CO_2 or air-quality gas sensors
- Add SMS or email alerts
- Add machine learning anomaly detection
- Make portable case for device